LAB - 03

QUESTION 1:
Display all integers within the closed interval [1,30] that satisfy the condition of being divisible by 2, ensuring that no additional values outside this specified domain are included in the output.

Sample Output:
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30

QUESTION 2:
Write a pseudocode that computes and displays the square of each integer within the closed interval [1,10], ensuring that the computation is performed iteratively.

Sample Output:
1 4 9 16 25 36 49 64 81 100

QUESTION 3:
Write a pseudocode that identifies and displays all integers within the inclusive range of 1 to 50 that are exactly divisible by 5.
Sample Output:
5 10 15 20 25 30 35 40 45 50

QUESTION 4:
Design an algorithm that continuously accepts integer inputs from the user until the sentinel value -1 is entered. After input termination, determine and display separately:

- the total number of even integers entered
- the total number of odd integers entered

Ensure that the sentinel value -1 is **not included** in either count.

Sample Input:
4 7 12 9 -1
Sample Output:
Total even numbers: 2
Total odd numbers: 2

QUESTION 5:
Write an algorithm that first accepts an integer N from the user, representing the total count of numbers to be entered. Then, read N integers and determine the **smallest** value among them. Finally, display this value.
Ensure your algorithm correctly handles any sequence of integers, including negative numbers and duplicates.

Sample Input:
6 15 -9 0 23 7 -3
Sample Output:
Smallest Number: -9

QUESTION 6:
Write an algorithm that asks the user to guess a secret number (for example: 8). The user has up to **5 tries** to guess it.
After each guess, if the number is correct, display "Correct Guess" and stop.
If the user does not guess correctly within 5 attempts, display "Game Over".

QUESTION 7:
Write an algorithm that accepts an **integer** from the user and counts the total number of **odd digits** present in that number.

Ensure your solution works correctly for:
positive numbers
negative numbers
numbers with repeated digits

Sample Input:
Enter a number: 472591
Sample Output:
Number of odd digits: 3

QUESTION 8:
Write an algorithm that reads a **non-negative integer** from the user and calculates its **factorial**. Ensure that your algorithm correctly handles any valid input, including 0 (whose factorial is 1).

Sample Input:
Enter a number: 5
Sample Output:
Factorial of 5 is : 120

QUESTION 9:
Write an algorithm to print a **right-aligned triangle of asterisks (*)** with 5 rows as shown below. Each row should have one more * than the previous row. Use **loops** to print the pattern.

Sample Output:
*
**
***
****
*****

QUESTION 10:
Write an algorithm that accepts an **integer** from the user and determines whether the number is a **palindrome** (i.e., it reads the same forwards and backwards).

(1) Sample Input:
Enter a number: 121
Sample Output:
121 is a palindrome.

(2) Sample Input:
Enter a number: 123
Sample Output:
123 is not a palindrome.

QUESTION 11:

The Endless Auction

Description An auction house is accepting bids for a rare artifact. The auctioneer reads out bids one by one. The bidding process stops only when the auctioneer sees the special "closing code" of -1.

Your job is to listen to the stream of bids and determine the **highest bid** placed during the session. If no valid bids are placed before -1, the highest bid is 0.

Input

- A series of positive integers representing bids.
- The sequence ends when the integer -1 is entered.

Output

- Print a single integer representing the largest number entered (excluding the -1).

Example

- **Input:**

100 50 250 90 -1

- **Output:** 250